

QuantEdge

A Backtesting Framework for Retail Quantitative Traders

Aaron Arauz [Baumender11] Luca Di Pietro [L-Di-Pietro]
Stefano Angelo Galdini [SAGaldini] Filippo Selmi [FilippoSelmi]

2026-06-12

USI Lugano — Programming in Finance II (2026)

Project 2.8 — Instructor: P. H. Gruber

Source repository: https://github.com/L-Di-Pietro/Programming_II_Project_USI

Web app: <https://quantedgeusi.up.railway.app/>

Abstract

QuantEdge is a web-based event-driven backtesting framework for retail quantitative traders, built as the final project for USI Lugano’s *Programming in Finance II* course. The system targets the two failure modes that cause most retail-backtest losses: look-ahead bias (architecturally prevented by enforcing $\text{bar-}t \rightarrow \text{bar-}t+1$ fills in the engine) and unrealistic execution assumptions (commission and slippage exposed as first-class user parameters). It ships eleven trading strategies across five families, multi-asset support (equities, ETFs, FX, crypto) with native trading calendars per asset class, both daily and hourly bar resolution, and a six-agent backend in which only two of the six agents are LLM-backed and are opt-in by default. This document covers the project plan, development diary, methodology, sample results with an economic reflection, lessons learned, and an acknowledgement of the generative-AI tools used.

Contents

1	Project plan	2
1.1	Problem statement	2
1.2	Scope	2
1.3	Tech-stack rationale	2
1.4	Four-week timeline	3
2	Project diary	3
3	Methodology: economics, mathematics, data science	5
3.1	Event-driven backtest semantics and look-ahead bias	5
3.2	Performance metrics	5
3.3	Annualisation factor by asset class and timeframe	6
3.4	Strategy taxonomy	6
3.5	The LLM-provider abstraction	7
4	Sample results and economic reflection	7
4.1	Run methodology	7
4.2	Results table	7
4.3	Economic reflection	8
4.4	Limitations of the demonstration	8
5	Lessons learned	8
5.1	What worked	8
5.2	What did not work	9
5.3	What we would do differently	9
5.4	What carries forward	9
6	Acknowledgement of generative-AI tools	9

1 Project plan

1.1 Problem statement

Retail quantitative traders lose money for two related reasons. First, many deploy strategies that have never been rigorously back-tested on historical data. Second, when they *do* test, they cut corners that systematically inflate apparent performance: they let the strategy peek at information that would not have been available in real time (*look-ahead bias*), they neglect transaction costs and execution slippage, and they evaluate over too few trades to distinguish edge from noise. The first problem is one of awareness; the second is a software-engineering problem. **QuantEdge** addresses the second: it is a backtesting platform that makes the rigorous path the easy one.

1.2 Scope

The system delivered in this submission (v1.0.0-rc1, tagged 2026-05-24) provides:

- Single-asset backtests across a seeded universe of 20 US equities, 5 ETFs, 5 crypto pairs and 6 FX pairs.
- Daily and hourly bar resolution, each indexed against the *native* trading calendar of its asset class (NYSE for equities and ETFs, 24×5 for FX, 24×7 for crypto).
- Eleven strategies across five families (trend, momentum, breakout, mean-reversion, benchmark), each implementing a uniform signal contract.
- An event-driven backtest engine that enforces look-ahead-bias prevention by construction.
- Configurable commission (bps), slippage (bps), position-sizing modes (fixed fraction or volatility-targeted), and an optional max-drawdown circuit breaker.
- Standard performance dashboard: equity curve, drawdown, monthly heatmap, trade-P&L scatter, rolling Sharpe, full KPI grid.
- Buy-and-hold and SPY benchmark overlays computed and persisted at run time.
- An optional, opt-in LLM-generated explanatory report grounded on the run’s metrics.

Out of scope for this iteration: multi-asset portfolio strategies, minute-level data, walk-forward optimisation UI, paper-trading or live-trading hooks. These are documented in the roadmap section of `README.md` as candidates for v1.1+.

1.3 Tech-stack rationale

The choices reflect a preference for tools that minimise the gap between the production-leaning quality the rubric expects and the four-week timeline the academic calendar allows:

- **Python 3.11+ / FastAPI / SQLAlchemy 2 / Pydantic v2** on the backend. FastAPI gives us OpenAPI for free (the live `/openapi.json` schema is the source of truth for the typed frontend client); SQLAlchemy’s generic types let the same schema run on SQLite in development and PostgreSQL in production via a single `DATABASE_URL` switch.
- **React 18 + TypeScript + Vite + TailwindCSS + Plotly.js** on the frontend: interactive charts without manual SVG plumbing, and compile-time schema checking against the OpenAPI-typed client.
- **pytest + Vitest** for tests. The single most important test is `tests/test_engine_no_lookahead.py`, which injects an oracle strategy that “knows” the future close — the engine must still only let it trade at next-bar open.
- **LLM via a provider abstraction:** `LLMProvider` with a deterministic `NullProvider` (used by tests) and an opt-in `GeminiProvider`; LLM features are off by default.

1.4 Four-week timeline

The project ran from 2026-04-25 (initial commit) to 2026-05-24 (documentation pass and submission), staffed by four contributors. Targets per week:

- **Week 1 — 2026-04-27 to 2026-05-03.** Scaffold the repository, fix the directory layout, draft `requirements.txt`, agree on the agent boundaries.
- **Week 2 — 2026-05-04 to 2026-05-10.** First domain feature (TSLA as an equity asset); first LLM-generated report (proves the `LLMProvider` abstraction works end-to-end).
- **Week 3 — 2026-05-11 to 2026-05-17.** Frontend MVP; hourly bar support and per-asset-class native calendars; chart suite; PDF export of the AI report.
- **Week 4 — 2026-05-18 to 2026-05-24.** Data-layer hardening (yfinance as primary across all classes); strategy catalogue expansion from 5 to 11; benchmark overlays; shared crosshair and legend; CI workflows; full documentation pass.

2 Project diary

The four-week development cycle was punctuated by eight merged pull requests and a handful of direct-to-main commits for docs and trivia; a further 25 PRs landed after submission, 15 of them opened automatically by the nightly documentation agent. The narrative below follows the actual order of decisions, with commit hashes given so a reviewer can verify any claim from `git log`.

Week 1 — Foundation (2026-04-25 to 2026-04-29)

The first commit (4ff85c1) bootstrapped an empty repository. Four days later, ecdf3bb (“Initial project scaffold for QuantEdge”) landed the directory layout we would carry through the project: `backend/` with `agents/`, `api/`, `analytics/`, `backtest/`, `data/`, `database/`, `llm/`, `strategies/` subpackages, a `frontend/` folder primed for Vite, `tests/`, and a pinned `requirements.txt`. The six agent boundaries (data, strategy, backtest, analytics, orchestrator, explanation) were committed to at this stage rather than discovered later, and held for the whole project.

Week 2 — First domain feature + first LLM call (2026-05-08)

Two same-day commits were the project’s two proofs of life. PR #1 (376d30e, “feat: add TSLA as new equity asset”) validated the end-to-end data path: seed an asset, run `load_initial_data.py`, see daily bars in the database. 8997d20 (“feat: add LLM generated report”) validated the LLM seam. We deliberately wrote the LLM feature *first* as a worst-case probe: if the `LLMProvider` abstraction could carry a real response, every deterministic feature after it would be easier. It worked, and the rest of the project never had to compromise its determinism for the sake of the AI report.

Week 3 — Frontend MVP, hourly bars, charts (2026-05-12 to 2026-05-16)

Frontend Version 1 (1f2f5c7) landed on 2026-05-12. The same day, PR #2 (1f13139, “Add native calendars and hourly bar support”) reshaped the data layer: the OHLCV primary key became (`asset_id`, `ts`, `timeframe`) so daily and hourly bars coexist, and the cleaner learned to reindex onto NYSE for equities, 24×5 for FX, and 24×7 for crypto — the project’s most intricate single change, since a calendar edge case would silently misstate every reported Sharpe ratio. A flurry of chart fixes followed (3e22d4c, 162ceba, ab1e2de), a3b6635 chunked bulk upserts past SQLite’s parameter ceiling and added LLM retries, and the first client-side PDF export shipped via jsPDF (0a7edb1).

Week 4 — CI hardening, data layer, strategies, benchmarks (2026-05-19 to 2026-05-22)

The week opened with two Claude Code GitHub Actions workflows: `claude.yml` (3d50b41) for @claude mentions and `claude-code-review.yml` (6a38f60) for an automatic review pass on every PR — both respond only to explicit human triggers and never self-merge. PR #4 (4ce0311, 820d610) added the `check-no-build-artifacts.yml` workflow after TypeScript compile outputs leaked into `frontend/src/`. The data layer consolidated on yfinance as primary source across all asset classes, with ccxt/Binance and Stooq

fallbacks (PR #5, c2b6cce); fixed-fractional sizing was corrected for crypto and FX (PR #6, ae082ef). PR #7 (b0676ed) doubled the strategy catalogue to eleven (MACD, Ichimoku, Keltner, Time Series Momentum, Stochastic, CCI), each registered, schema-exposed, and unit-tested on a known fixture. PR #8 (c0801c7) added the benchmark overlays: same-asset and SPY buy-and-hold curves computed and persisted at run time, toggleable from a single shared chart legend. d1efa3f (Quick-Start docs) and 4a9e6ae (yfinance 0.2.46 → 1.3.0, which pulled in curl_cffi) closed the week.

Documentation pass (2026-05-24)

79e200a refreshed the existing docs against the final code state, and a second same-day pass created the artefacts the rubric requires, this PDF among them: AGENTS.md, CHANGELOG.md, CONTRIBUTING.md, CITATIONS.md, AI_USAGE.md, SECURITY.md, CODE_OF_CONDUCT.md, docs/user-guide.md, docs/api.md, docs/architecture-diagrams/, and the docs/academic/ bundle.

Post-submission automation and hardening (2026-05-24 to 2026-06-07)

On submission evening, PR #9 (612e39e) introduced `nightly-docs.yml`: a GitHub Actions job that wakes at 02:00 UTC, runs Claude Code headless with a read-only Bash allowlist, and opens a PR whenever it finds documentation drift against the last 24 hours of commits — it never touches `main` directly. 4dfc948 and PR #10 (58a4811) fixed its authentication and `id-token: write` permission; PR #12 (01d9c30) bumped all workflows to the Node-24 action majors ahead of GitHub's runner retirement.

Substantive engineering continued alongside. PR #16 (ed31a92, b8c9332, 533dedb) hardened `run.py`: TTY detection before the API-key prompt, a fail-fast port check with a `kill` hint, and tolerant `.env` parsing. e1dd918 added a production frontend Dockerfile and Postgres URL normalisation for Railway deployment. On 2026-05-29 a frontend polish sprint (b3acd7d, PR #18, 45dddf4, 6056272) shipped the interactive onboarding tour, a unified loading screen, and run-detail prefetching, while PR #19 (38816c3, fd57ede) made the downloadable report a self-contained interactive HTML file. The same day, 1acc780 introduced per-browser backtest isolation: an anonymous `client_id` minted in `localStorage` and forwarded as `X-Client-Id` scopes `GET /backtests` to the caller, with an integration test asserting two clients cannot see each other's runs.

PR #21 (b7996f6, 2026-06-01) replaced the jsPDF export with a print-quality pipeline: the frontend assembles the self-contained report HTML and POSTs it to `/backtests/{run_id}/report.pdf`, where headless Chromium (Playwright) renders A4 output identical to the interactive report. b6718ce pinned the backend image to `python:3.11-slim-bookworm` after a floating tag silently resolved to Debian 13 and broke Chromium's font dependencies. b9414a7 added mobile-first navigation, and 1ebcae4 (2026-06-03) fixed print sizing by relaying out every Plotly figure to the A4 column width before `window.print()`. 855c0a6 (2026-06-05) repaired the missing `\keystroke` macro this document uses and completed the author list. Finally, PR #30 (2026-06-07) added the `MobileDesktopBanner` and suppressed the onboarding tour below the desktop breakpoint (7ca67e8), rebranded the project from QuantBacktest to QuantEdge across every file (da11eaf), and moved the port check to the second step of `run.py` so conflicts abort in under a second (a0b8ac2).

Steady state and final pass (2026-05-25 to 2026-06-11)

From 2026-05-25 onward the nightly workflow fired every night, producing 15 merged PRs through #33. On nights following human commits it recorded them in `CHANGELOG.md`, this diary, and the user guide; on quiet nights it recorded only the previous nightly commit itself. No nightly run ever modified code, confirming that the documentation-only write scope and read-only Bash allowlist behave as designed. A final pre-submission pass on 2026-06-11 populated Table 2 from a backtest executed against the final codebase, captured the user-guide screenshots from a live local instance, compressed this diary — which nightly appends had grown well past the rubric's page budget — and recompiled this PDF.

Things we tried and abandoned

A few decisions did not survive the four weeks:

- **Asynchronous backtest submission.** The first design queued runs and polled status via WebSocket. We backed it out because synchronous POST worked well enough at v1 scale and removed an entire layer of state to debug.
- **Postgres in development from day one.** We initially set up Docker Compose with Postgres but it slowed local iteration noticeably. SQLite became the default and the schema was kept on SQLAlchemy generic types so a `DATABASE_URL` swap reaches Postgres without code changes.

- **LLM tool-use via the provider SDK’s native function calling.** The Orchestrator Agent parses tool calls from a JSON envelope in the model output rather than relying on Gemini’s native function-calling API, because the latter is not portable to other providers.

3 Methodology: economics, mathematics, data science

This section makes the project’s analytical contract precise: the event-driven backtest semantics, the KPI formulas with their per-asset-class annualisation factor, the strategy taxonomy, and the LLM-provider abstraction.

3.1 Event-driven backtest semantics and look-ahead bias

Look-ahead bias is the canonical pitfall of backtest construction: a strategy “sees” information that, in real time, it would not have had access to when it needed to act [10, 3]. The most common form is a *single-bar* leak — a strategy reads bar t ’s closing price and then places an order that fills at bar t ’s open or close. Across a long backtest the false edge accumulates and inflates every downstream metric. QuantEdge forbids the leak architecturally: orders generated during bar t fill at the *open of bar $t+1$* , with commission and slippage applied to the fill price. The rule is implemented once, in `backend/backtest/engine.py`, and asserted by an oracle-strategy test in `tests/test_engine_no_lookahead.py` that injects a strategy that “knows” the future close; the test passes only if the engine still forces the oracle to trade at next-bar open.

In pseudocode the engine loop is:

```
for t in range(len(bars)):
    # Target position from data up to and including bar t.
    target = strategy.generate_signals(bars[:t+1]).iloc[t] # in {-1, 0, 1}
    if t == len(bars) - 1: break # no t+1 open to fill at
    delta = target - portfolio.current_position
    if delta == 0: continue
    # Fill at bar (t+1) open, applying slippage and commission.
    fill_price = bars.iloc[t+1].open * (1 + sign(delta) * slippage_bps / 10_000)
    commission = abs(delta * portfolio.size) * fill_price * commission_bps / 10_000
    portfolio.apply(delta, fill_price, commission)
```

The signal contract is uniformly `pd.Series[int]` with values in $\{-1, 0, +1\}$ for “short, flat, long”. Strategies do not multiply their own signals by a position-size factor; the engine handles sizing via either *fixed fractional* (a fraction of equity per trade) or *volatility targeting* (position scaled inversely to realised volatility). An optional circuit-breaker forces the run flat if equity draws down beyond a user-supplied threshold.

3.2 Performance metrics

The KPI definitions follow the conventions in Bacon [2]. Let $\{r_t\}_{t=1}^T$ denote the per-bar returns of the equity curve $\{V_t\}_{t=0}^T$, k the number of bars in one calendar year (Section 3.3), and $y = T/k$ the run’s length in years.

Total return and CAGR.

$$R = \frac{V_T}{V_0} - 1, \quad \text{CAGR} = \left(\frac{V_T}{V_0} \right)^{1/y} - 1. \quad (1)$$

Annualised volatility.

$$\sigma_{\text{ann}} = \sqrt{k} \sigma(r_t). \quad (2)$$

Sharpe and Sortino ratios.

$$\text{Sharpe} = \sqrt{k} \frac{\bar{r}}{\sigma(r_t)}, \quad \text{Sortino} = \sqrt{k} \frac{\bar{r}}{\sigma_{-}(r_t)}, \quad (3)$$

where $\sigma_{-}(r_t)$ is the downside semi-deviation $\sqrt{\mathbb{E}[\min(r_t, 0)^2]}$. We adopt the zero-risk-free-rate convention for retail backtesting [13, 14].

Maximum drawdown and Calmar ratio.

$$\text{maxDD} = \min_t \frac{V_t - \max_{s \leq t} V_s}{\max_{s \leq t} V_s}, \quad \text{Calmar} = \frac{\text{CAGR}}{|\text{maxDD}|}. \quad (4)$$

Calmar [17] captures the pain-adjusted return: how much annual growth the strategy delivered per unit of drawdown the user had to endure.

Trade-level metrics. Let $\{p_i\}_{i=1}^N$ denote per-leg net P&L:

$$\text{winrate} = \frac{|\{i : p_i > 0\}|}{N}, \quad \text{PF} = \frac{\sum_{p_i > 0} p_i}{\sum_{p_i < 0} |p_i|}, \quad (5)$$

$$\text{expectancy} = \text{winrate} \cdot \overline{p_+} - (1 - \text{winrate}) \cdot |\overline{p_-}|, \quad (6)$$

where $\overline{p_+}$ and $\overline{p_-}$ are the means of the positive and negative P&L populations respectively [12, 15].

3.3 Annualisation factor by asset class and timeframe

A Sharpe ratio computed against the wrong k misstates the annualised number by tens of percent. QuantEdge looks up k from `backend/analytics/periods.py` as a function of `(asset_class, timeframe)`:

Asset class	Daily k	Hourly k
Equity / ETF (NYSE)	252	1 638
FX (24×5)	260	6 240
Crypto (24×7)	365	8 760

Table 1: Annualisation factor k per asset class and timeframe. NYSE hourly uses 252×6.5 elapsed market hours per session.

The NYSE hourly value reflects elapsed market hours, not the literal bar count, because the first hourly bar of a regular session covers only the opening 30-minute auction. Early-close sessions (about ten per year) further reduce the count slightly; the lookup is calibrated against the `exchange_calendars` XNYS schedule rather than computed analytically.

3.4 Strategy taxonomy

The eleven shipped strategies populate five families. Each family responds to a different market regime; running them side-by-side makes that regime sensitivity visible, which is the explicit pedagogical motive for shipping the breadth.

- **Trend** (SMA Crossover [12], MACD Crossover [1], Ichimoku Cloud [6]): designed to capture sustained directional moves.
- **Momentum** (Time Series Momentum [11]): own-asset trailing-return sign; profits from 1–12 month persistence.
- **Breakout** (Donchian [5], Keltner [7]): long on channel breaks; trades clean trending regimes.
- **Mean reversion** (Bollinger [4], CCI [8], RSI [16], Stochastic Oscillator [9]): trade fades within range-bound regimes; bleed in trends.
- **Benchmark** (Buy and Hold): always long; serves as the reference both for active strategies and for the on-chart same-asset overlay.

All eleven strategies share the same signal contract and therefore the same engine path. Adding a twelfth is a single-file change plus a registry edit; the frontend’s parameter form regenerates from the new JSON Schema.

3.5 The LLM-provider abstraction

Two of the six runtime agents — the Orchestrator and the Explanation Agent — are LLM-backed, and every call they make goes through `LLMProvider.generate(...)`. Two implementations ship: `NullProvider` (deterministic canned text) and `GeminiProvider` (Google’s Gemini API). The abstraction earns its keep three ways: tests use `NullProvider` exclusively and stay reproducible without an API key; switching provider is a config change, not a code change; and it makes the project’s policy explicit — *the deterministic surface is the default*, and any non-determinism is a conscious operator opt-in (`LLM_ENABLED=true` plus a populated `GEMINI_API_KEY`).

4 Sample results and economic reflection

This section reports a representative backtest, contextualises it against the benchmark overlay the system computes automatically, and reflects on what the result implies about the strategy’s regime sensitivity. The numeric cells in Table 2 are taken from a run executed against the final codebase on 2026-06-11, so the table represents the actual code path rather than a fabricated example.

4.1 Run methodology

Configuration. We report a backtest of the `sma-crossover` strategy on SPY (SPDR S&P 500 ETF Trust), over 2020-01-01 to 2024-12-31, daily bars (`timeframe="1d"`). Strategy parameters: `fast_window=20`, `slow_window=50`, `allow_short=false`. Execution parameters: `initial_cash=10 000`, `commission_bps=5`, `slippage_bps=2`, `sizing_mode="fixed_fraction"`, `risk_fraction=1.0`, no max-DD circuit breaker. The annualisation factor is $k = 252$ (NYSE daily, Section 3.3).

Reproducibility. The exact API call:

```
curl -s -X POST http://127.0.0.1:8000/backtests -H 'Content-Type: application/json' \
-d '{"asset_symbol": "SPY", "strategy_slug": "sma-crossover",
    "start_date": "2020-01-01T00:00:00", "end_date": "2024-12-31T00:00:00",
    "params": {"fast_window": 20, "slow_window": 50, "allow_short": false},
    "timeframe": "1d"}'
```

Then `GET /backtests/{run_id}/metrics` and `GET /backtests/{run_id}/benchmark/buy_and_hold/metrics` supply the table values. Because the traded asset here *is* SPY, the same-asset and S&P 500 overlays would coincide, so `BacktestAgent` deliberately persists only the former; runs on any other asset carry both.

4.2 Results table

Metric	SMA Crossover	Buy-and-hold (SPY)
Total return	51.2%	96.6%
CAGR	8.6%	14.5%
Annualised vol	12.5%	21.0%
Sharpe	0.73	0.75
Sortino	1.08	1.16
Max drawdown	−27.9%	−33.7%
Max drawdown duration	1 093 days	173 days
Calmar	0.31	0.43
Trade count	25	—
Win rate	20.0%	—
Profit factor	2.20	—

Table 2: KPIs for the representative SMA-crossover run, alongside the buy-and-hold benchmark overlay computed automatically by `BacktestAgent`. Trade-block cells are dashed for the benchmark: the overlay is an equity curve, not a traded portfolio, so trade statistics are not computed for it.

4.3 Economic reflection

The economic story in Table 2 follows from the nature of the strategy and the chosen window. SMA Crossover is a trend-follower: it goes long when the 20-day SMA crosses above the 50-day SMA and exits when it crosses back. The 2020–2024 window spans the COVID drawdown (March 2020), the equity bull market of 2020–2021, the 2022 bear market, and the 2023–2024 recovery — four distinct regimes inside five years. Three observations stand out from the actual run:

1. **The drawdown-protection thesis only half-delivered.** The strategy captured about half of buy-and-hold’s total return (51.2% vs. 96.6%) but its maximum drawdown was only modestly shallower (−27.9% vs. −33.7%), so buy-and-hold wins the pain-adjusted comparison (Calmar 0.43 vs. 0.31). The culprit is 2022: the bear market was choppy rather than cleanly trending, and bear-market rallies repeatedly pulled the crossover back in just in time for the next leg down. The strategy’s 2022 calendar return was −23.2% — *worse* than SPY’s own −18.2% over the same year — in precisely the regime a trend-follower is meant to sidestep.
2. **Per-unit-risk efficiency was preserved, not improved.** The Sharpe ratios are nearly identical (0.73 vs. 0.75) at much lower volatility (12.5% vs. 21.0%): over this window the crossover behaved like a de-levered, smoother version of the index rather than an alpha source. The trade ledger shows the classic trend-following shape — a 20% win rate combined with a 2.20 profit factor, i.e. many small whipsaw losses paid for by a few large trend captures (average win $\approx 8.8\times$ average loss).
3. **The result is statistically and psychologically fragile.** Twenty-five trades over five years is far below the ~ 100 -trade rule-of-thumb for statistical significance, and the strategy spent 1 093 days — roughly three of the five years — below its own high-water mark (vs. 173 days for buy-and-hold). A backtest that ends in profit can still be untradeable in practice if the user abandons it mid-drawdown.

4.4 Limitations of the demonstration

The result above is a single asset over a single window with a single parameter set. It is illustrative, not predictive. Three caveats apply:

- **Survivorship bias** — yfinance carries only currently listed tickers, so any equity universe is biased upward.
- **Single-window evaluation** — the rubric’s preference for walk-forward / out-of-sample evaluation is correct; SMA-crossover’s in-sample fit on a single window does not establish edge.
- **The README’s retail-trader tip applies** — if the backtested maximum drawdown is 20%, the user should mentally prepare for 30% live, since regime change and realistic retail slippage are both worse than the backtest assumes.

5 Lessons learned

5.1 What worked

Pydantic-driven JSON Schemas as the contract between backend and frontend. Every strategy in `backend/strategies/` declares a Pydantic `Config` class; `GET /strategies` serialises it as a JSON Schema; the React form component renders a complete parameter UI from that schema with no per-strategy frontend code. Adding the sixth-through-eleventh strategies (PR #7) cost zero frontend work as a result — the single biggest productivity multiplier in the project.

The six-agent abstraction. Splitting the runtime into named agents, each inheriting `BaseAgent[TIn, TOut]` with a single public `run(payload)` method, gave us seam-by-seam testability and a natural docking point for the LLM orchestrator. The boundary between the two LLM-backed agents and the four deterministic agents was the right boundary: it kept the test suite reproducible while leaving the LLM path open as an opt-in feature.

The $\{-1, 0, +1\}$ signal contract. Forbidding strategies to manage their own position size, commissions, or fills made the engine the single point of truth for the $\text{bar-}t \rightarrow \text{bar-}t+1$ rule, and meant that the look-ahead-bias oracle test in `tests/test_engine_no_lookahead.py` is the only invariant guard we need. Every strategy passes through the same fill code path; a bug anywhere in fill semantics surfaces everywhere at once.

Native trading calendars per asset class. The temptation to reindex every asset onto NYSE for “consistency” would have silently misstated the annualisation factor for FX and crypto by 5–50%. Building calendar-aware reindexing into the cleaner from the start (PR #2) cost a few hours and saved us from a class of subtle correctness bugs.

5.2 What did not work

Yahoo Finance’s 730-day hourly cap. The cap is a hard external constraint that propagates into the UI (date pickers had to gain a per-timeframe maximum-look-back constraint) and into the data layer (the Crypto Fetcher learned to fall back to Binance via `ccxt`). We absorbed the cap; the alternatives (paid intraday data, scraping) cost more than the limitation imposes.

LLM tool-call JSON parsing. The Orchestrator Agent parses tool calls out of the model’s response by extracting a JSON envelope. This is fragile: models occasionally wrap the JSON in markdown fences or prose. We chose provider-portability over robustness for v1; a future iteration could fall back to provider-native function calling where available.

Asynchronous backtest execution. The first design had submission return immediately and the UI poll for status; we backed it out to a synchronous `POST /backtests` because runs on daily bars complete in under a second and the async machinery added an entire state-management layer that nothing in v1 actually needed (Section 2).

5.3 What we would do differently

- **Walk-forward UI from day one.** The Strategy Agent already exposes a chronological train/test split, but the UI does not surface it. Without a UI affordance, users will tune parameters by re-running the full backtest — the canonical recipe for overfitting [3]. The UI for walk-forward should have been a v1 feature, not a v1.1 candidate.
- **Postgres as the default development database.** SQLite was fast to set up but does not catch the subset of bugs that only show up under concurrent writes or under strict referential integrity. The schema is already Postgres-compatible (SQLAlchemy generics only); next time we would default to Postgres and let users opt down to SQLite.
- **Earlier CI hygiene.** The `check-no-build-artifacts.yml` workflow should have been the first `.github/workflow` we wrote, not the third. Build-artefact leakage produced noisy diffs for a week before we formalised the guard.
- **Smaller, more frequent PRs.** The 32 development-cycle commits landed via only 8 merged PRs; some touched a dozen files across two subsystems. Reviewers (human or AI) work better on small, focused diffs.

5.4 What carries forward

The patterns we would copy verbatim into the next project: the schema-as-API discipline, the agent abstraction with a single public `run` method, the deterministic-by-default LLM stance with `NullProvider` for tests, the per-asset-class trading-calendar layer, the $\{-1, 0, +1\}$ signal contract, and the “*Last verified against code*” footer convention on every documentation file.

6 Acknowledgement of generative-AI tools

This section condenses `AI_USAGE.md` at the repository root, which carries the full per-tool table and policy statement.

Team policy

The team used generative-AI tools as collaborators, not as authors. Three durable rules governed every contribution:

1. Every line of code that lands in `main` is reviewed by a human. AI output that no team member can explain is not merged.

2. No AI tool is given access to credentials, API keys, or production data; only the secret-free `.env.example` ships in the repository.
3. The LLM features inside the product are opt-in: `LLM_ENABLED` defaults to `false`, tests use the deterministic `NullProvider`, and the live `GeminiProvider` requires explicit configuration.

We take collective responsibility for the whole repository, including text and code originally drafted by an AI tool.

Tools used

Claude Code (Anthropic, CLI agent) was the primary coding assistant for multi-file refactors, agent scaffolding, documentation drafts, and the bulk of the LaTeX source for this submission; its repository footprints are the `claude.yml`, `claude-code-review.yml`, and `nightly-docs.yml` workflows. **Claude.ai** (Anthropic, web) was used by team members for design discussions and prose drafting. **ChatGPT** (OpenAI) was used opportunistically for isolated syntax checks and LaTeX-table formatting. **GitHub Copilot** (GitHub/OpenAI) provided in-editor autocomplete for routine code; one merge-resolution commit (`b9fc033`) was authored by the `copilot-swe-agent[bot]` account. **Cursor** (Anysphere) was used by one team member as their primary editor for some of the frontend work. **Google Gemini** (web) was used to spot-check factual claims about NYSE hours and `ccxt` pagination semantics; the Gemini API wired into the product itself is a product feature, not a development tool.

Where AI was not used

The strategy formulas in `backend/strategies/*.py` were transcribed from the sources cited in `CITATIONS.md` and Section 3 and independently re-derived by the team. The look-ahead-bias enforcement in `backend/backtest/engine.py` and its oracle-strategy test, the database schema, the API contract, and the four-week timeline were team work, not AI-generated. A final caveat: AI sessions are not natively reproducible; we mitigate via diff-level human review, the test suite’s invariant coverage, and prompt summaries in the project diary.

7 References

- [1] Gerald Appel. *Technical Analysis: Power Tools for Active Investors*. Upper Saddle River, NJ: FT Press, 2005. ISBN: 978-0131479029.
- [2] Carl R. Bacon. *Practical Portfolio Performance Measurement and Attribution*. 2nd ed. Chichester, UK: Wiley, 2008. ISBN: 978-0470059289.
- [3] David H. Bailey and Marcos López de Prado. “The Probability of Backtest Overfitting”. In: *Journal of Computational Finance* 20.4 (2014), pp. 39–69. DOI: [10.21314/JCF.2016.322](https://doi.org/10.21314/JCF.2016.322).
- [4] John Bollinger. *Bollinger on Bollinger Bands*. New York, NY: McGraw-Hill, 2001. ISBN: 978-0071373685.
- [5] Curtis M. Faith. *The Way of the Turtle: The Secret Methods that Turned Ordinary People into Legendary Traders*. New York, NY: McGraw-Hill, 2007. ISBN: 978-0071486644.
- [6] Goichi Hosoda. *Ichimoku Kinkō Hyō*. Tokyo, Japan: Self-published, 1969.
- [7] Chester W. Keltner. *How to Make Money in Commodities*. Kansas City, MO: Keltner Statistical Service, 1960.
- [8] Donald R. Lambert. “Commodity Channel Index: Tool for Trading Cyclic Trends”. In: *Commodities Magazine* (Oct. 1980).
- [9] George C. Lane. “Lane’s Stochastics”. In: *Technical Analysis of Stocks & Commodities* 2.3 (1984).
- [10] Marcos López de Prado. *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley, 2018. ISBN: 978-1119482086.
- [11] Tobias J. Moskowitz, Yao Hua Ooi, and Lasse Heje Pedersen. “Time Series Momentum”. In: *Journal of Financial Economics* 104.2 (2012), pp. 228–250. DOI: [10.1016/j.jfineco.2011.11.003](https://doi.org/10.1016/j.jfineco.2011.11.003).
- [12] Robert Pardo. *The Evaluation and Optimization of Trading Strategies*. 2nd ed. Hoboken, NJ: Wiley, 2008. ISBN: 978-0470128015.
- [13] William F. Sharpe. “Mutual Fund Performance”. In: *The Journal of Business* 39.1 (1966), pp. 119–138.

- [14] Frank A. Sortino and Lee N. Price. “Performance Measurement in a Downside Risk Framework”. In: *The Journal of Investing* 3.3 (1994), pp. 59–64. DOI: [10.3905/joi.3.3.59](https://doi.org/10.3905/joi.3.3.59).
- [15] Van K. Tharp. *Trade Your Way to Financial Freedom*. New York, NY: McGraw-Hill, 1998. ISBN: 978-0070647626.
- [16] J. Welles Wilder. *New Concepts in Technical Trading Systems*. Greensboro, NC: Trend Research, 1978. ISBN: 978-0894590276.
- [17] Terry W. Young. “Calmar Ratio: A Smoother Tool”. In: *Futures* 20.1 (1991).